LeafLens

AI-Powered Leaf Disease Prediction System

Architecture Document

Abhijeet Kumar (DA24B001)

1 Introduction

Plant diseases can affect crop quality and agricultural productivity. Identifying a disease early gives farmers and other users a better opportunity to investigate the problem and take appropriate action.

LeafLens was developed to make image-based plant disease identification more accessible. A user uploads a photograph of a leaf, and the application uses a trained deep learning model to predict its disease category.

The project goes beyond training a classification model. It also includes data processing, experiment tracking, API-based model serving, a user interface, containerised deployment, and application monitoring.

This document explains how these pieces fit together and how data moves through the complete system.

1.1 Main Objectives

The main objectives of LeafLens are:

- Classify plant leaf images into 38 disease and healthy-leaf categories.
- Maintain a reproducible data preparation and model training workflow.
- Track experiments, evaluation metrics, and model artifacts.
- Provide predictions through a REST API.
- Offer a straightforward interface for users.
- Monitor application performance and failures.
- Package the application and monitoring services using Docker Compose.

The current system is designed as a decision-support application. Its predictions should not replace expert agricultural diagnosis.

2 Overall System Architecture

LeafLens has two main workflows. The first is the **offline ML workflow**, where the dataset is prepared, the model is trained and its performance is evaluated. The second is the **online prediction workflow**, where a user submits an image and receives a prediction.

Keeping these workflows separate makes the system easier to maintain. Model development can take place without changing the user interface, while the application can serve predictions without running the training pipeline.

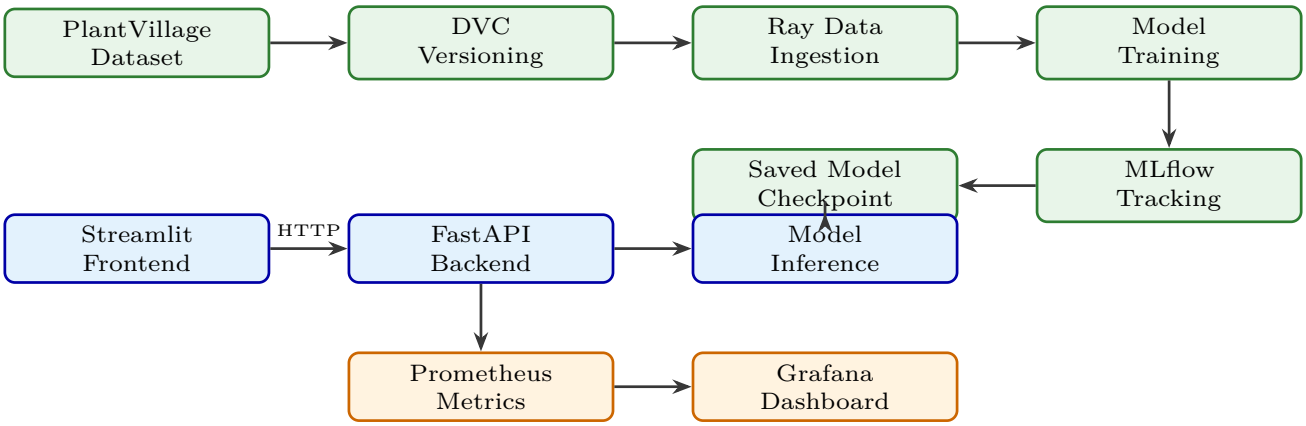


Figure 1: Overall architecture of LeafLens

The offline workflow produces a trained model checkpoint. That checkpoint is loaded by the inference component used by the FastAPI backend. The frontend communicates with the backend through HTTP requests, while Prometheus and Grafana observe the running application.

3 Major Components

The application is divided into modules with clear responsibilities. This avoids placing data processing, prediction logic, user interaction and monitoring in one large application component.

Table 1: Major components and their responsibilities

Component	Technology	Responsibility
Dataset	PlantVillage	Supplies labelled leaf images.
Versioning	DVC	Tracks dataset and pipeline outputs.
Ingestion	Ray Data	Loads and validates image data.
Training	PyTorch	Trains the image classification model.
Experiment tracking	MLflow	Records parameters, metrics and artifacts.
Backend	FastAPI	Handles uploads, validation and prediction requests.
Inference	PyTorch	Loads the trained checkpoint and predicts classes.
Frontend	Streamlit	Provides the image upload and result interface.
Monitoring	Prometheus	Collects application metrics.
Dashboard	Grafana	Displays application and inference behaviour.
Deployment	Docker Compose	Runs the application services together.

3.1 Frontend and Backend

The frontend is built using Streamlit. It allows users to upload an image and view the resulting prediction without needing to interact with the model directly.

FastAPI acts as the interface between the frontend and the machine learning model. It receives the uploaded file, validates it, calls the prediction service and returns a structured response.

The frontend does not load the PyTorch model itself. Instead, it communicates with the backend through an HTTP API. This keeps the interface independent of the model implementation.

3.2 Prediction Component

The inference component uses MobileNetV3-Large, fine-tuned for the 38 target classes. It loads the saved checkpoint, applies the required image transformations and calculates class probabilities.

The class with the highest predicted probability is returned along with its confidence value.

3.3 Monitoring Components

Prometheus collects metrics exposed by the backend through its `/metrics` endpoint. Grafana uses Prometheus as its data source and presents the collected information in a monitoring dashboard.

The dashboard includes backend availability, request rate, prediction count, prediction confidence, API latency, HTTP errors and prediction failures.

4 Dataset and Model Architecture

4.1 Dataset

LeafLens uses the PlantVillage dataset, containing 54,305 images across 38 plant disease and healthy-leaf classes.

A stratified split was used so that the class distribution is represented across the training, validation and test subsets.

Table 2: Dataset distribution

Subset	Images
Training	38,013
Validation	8,146
Testing	8,146
Total	**54,305**

The images are originally 256 × 256 pixels. During model training and inference, they are resized to 224 × 224 pixels.

4.2 Classification Model

MobileNetV3-Large was selected as the model architecture. It uses pretrained ImageNet weights and is fine-tuned for the plant disease classification task.

The model configuration is summarised below.

Table 3: Training configuration

Parameter	Value
Architecture	MobileNetV3-Large
Pretrained weights	ImageNet
Input resolution	224×224
Batch size	32
Epochs	5
Optimizer	AdamW
Learning rate	0.0001
Weight decay	0.0001
Loss function	CrossEntropyLoss
Output classes	38

4.3 Test Results

The final model was evaluated on the held-out test set.

Table 4: Classification performance on the test set

Metric	Result
Test samples	8,146
Correct predictions	8,108
Incorrect predictions	38
Accuracy	99.53%
Macro Precision	99.27%
Macro Recall	99.34%
Macro F1-score	99.28%

These results describe performance on the PlantVillage test split. Since the dataset contains controlled images, the same performance should not be assumed for photographs taken in varied outdoor conditions.

5 Offline Machine Learning Workflow

The offline workflow covers dataset preparation, ingestion, training, evaluation, and model storage.

DVC is used to track the dataset and pipeline stages. Ray Data handles image ingestion and validation, while MLflow records the training experiments.

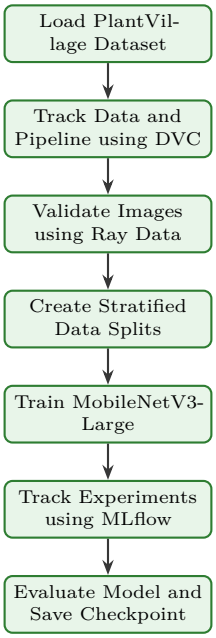


Figure 2: Offline machine learning workflow

5.1 Data Ingestion

Ray Data was used to process the dataset and validate the image records. The completed run processed all 54,305 images without reporting invalid images.

Table 5: Ray Data ingestion results

Metric	Result
Images processed	54,305
Valid images	54,305
Invalid images	0
CPU workers	2
Data blocks	8
Batch size	128
Execution time	7.553 seconds
Throughput	7,189.73 images/second

The ingestion workflow also generates a Parquet metadata artifact containing information about the processed images.

5.2 Experiment Tracking

MLflow maintains the record of each training experiment. The logged information includes model parameters, epoch-level metrics, evaluation results, and model artifacts.

This gives the project a clear record of how a model was trained and which experiment produced a particular checkpoint.

6 Online Prediction Workflow

The online workflow begins when a user uploads a leaf image through the Streamlit interface.

The request reaches FastAPI, where the backend checks whether the uploaded file is valid. If validation succeeds, the image is converted to RGB and passed to the inference component.

The model calculates the class probabilities and returns the predicted disease category and confidence. The backend then sends the response to the frontend for display.

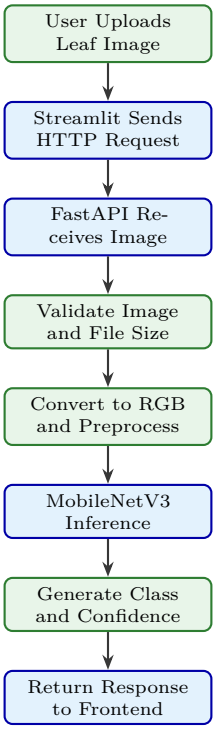


Figure 3: Online prediction workflow

6.1 Input Validation and Error Handling

The backend performs validation before inference. The current implementation checks for:

- Empty uploads.
- Files larger than the configured 10 MB limit.
- Invalid or unreadable image content.
- Image conversion failures.

Invalid requests are rejected rather than passed to the model. Prediction failures are logged and counted through the monitoring metrics.

7 Monitoring and Deployment

7.1 Monitoring Architecture

The backend exposes application metrics through the `/metrics` endpoint. Prometheus scrapes these metrics at regular intervals, and Grafana presents them through a dashboard.

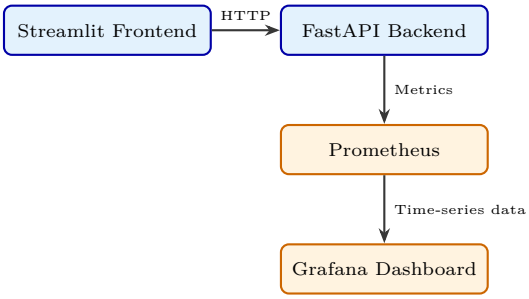


Figure 4: Application monitoring workflow

The dashboard provides visibility into:

- Backend availability and HTTP request rate.
- Total predictions and prediction confidence.
- P95 API latency.
- HTTP errors and prediction failures.
- Predictions grouped by disease class.

7.2 Containerised Deployment

Docker Compose runs the application as four separate services. Each service has its own responsibility.

Table 6: Deployment configuration

Service	Technology	Host Port
Backend	FastAPI	8001
Frontend	Streamlit	8501
Prometheus	Prometheus	9092
Grafana	Grafana	3002

The frontend communicates with the backend using its internal Docker service address. The backend health check allows Docker Compose to verify that the API is ready before the frontend starts.

8 Architectural Decisions and Limitations

The architecture was designed to keep the application simple to operate while still covering the main stages of an AI application lifecycle.

Table 7: Key architectural decisions

Decision	Reason
Separate frontend and backend	Allows the user interface and prediction logic to evolve independently.
REST-based inference	Provides a clear interface between the frontend and model serving.
DVC and MLflow	Support dataset, pipeline, experiment, and artifact tracking.
Prometheus and Grafana	Make application behaviour visible after deployment.
Docker Compose	Provides a straightforward way to run the complete local system.

There are also some limitations worth keeping in mind. The model is trained and evaluated using PlantVillage images, which may not capture the full variety of real field conditions. The current monitoring setup focuses on application performance and inference activity; it does not implement a complete model-drift detection system.

The Docker Compose deployment is intended for project demonstration and local use, rather than as a complete large-scale production infrastructure.

9 Conclusion

LeafLens brings together image classification, data engineering, experiment tracking, API-based inference, and application monitoring in one system.

The separation between model development and online prediction makes the application easier to maintain, while the monitoring layer provides visibility into how the deployed system behaves.

The goal was not simply to train a model that produces accurate predictions, but to build an application around that model which can be tested, operated, and understood.